

UNIFIED MULTIMODAL RL

UniRL

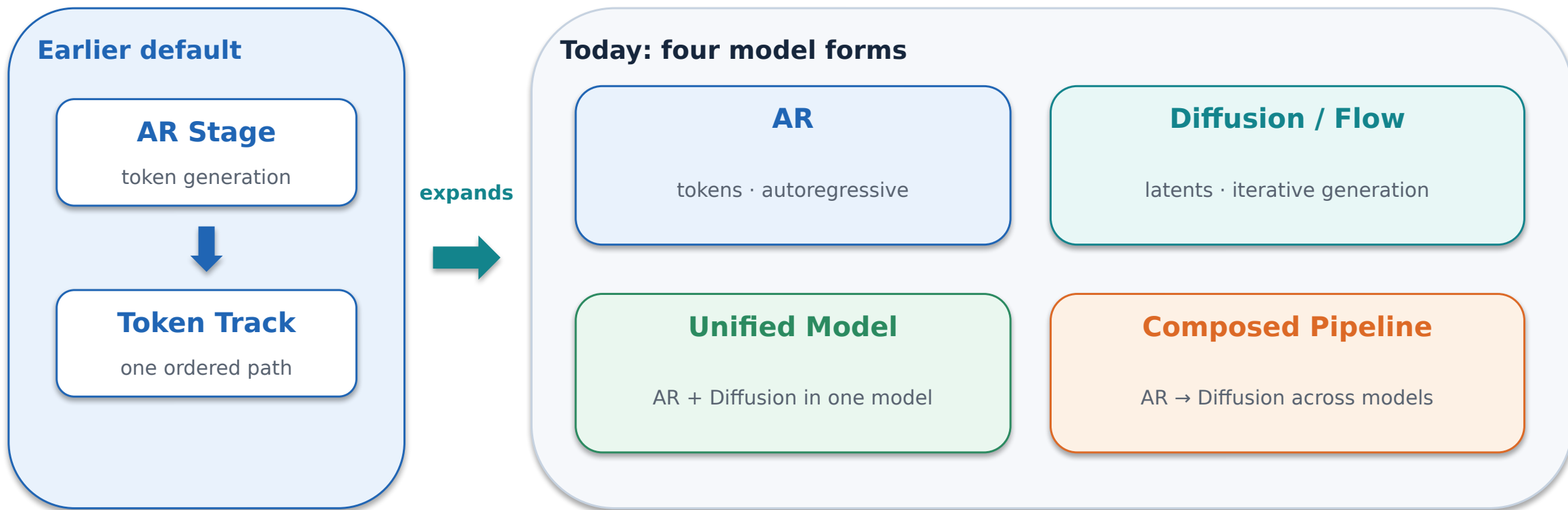
One Loop, Any Modality

金乐伟

腾讯混元

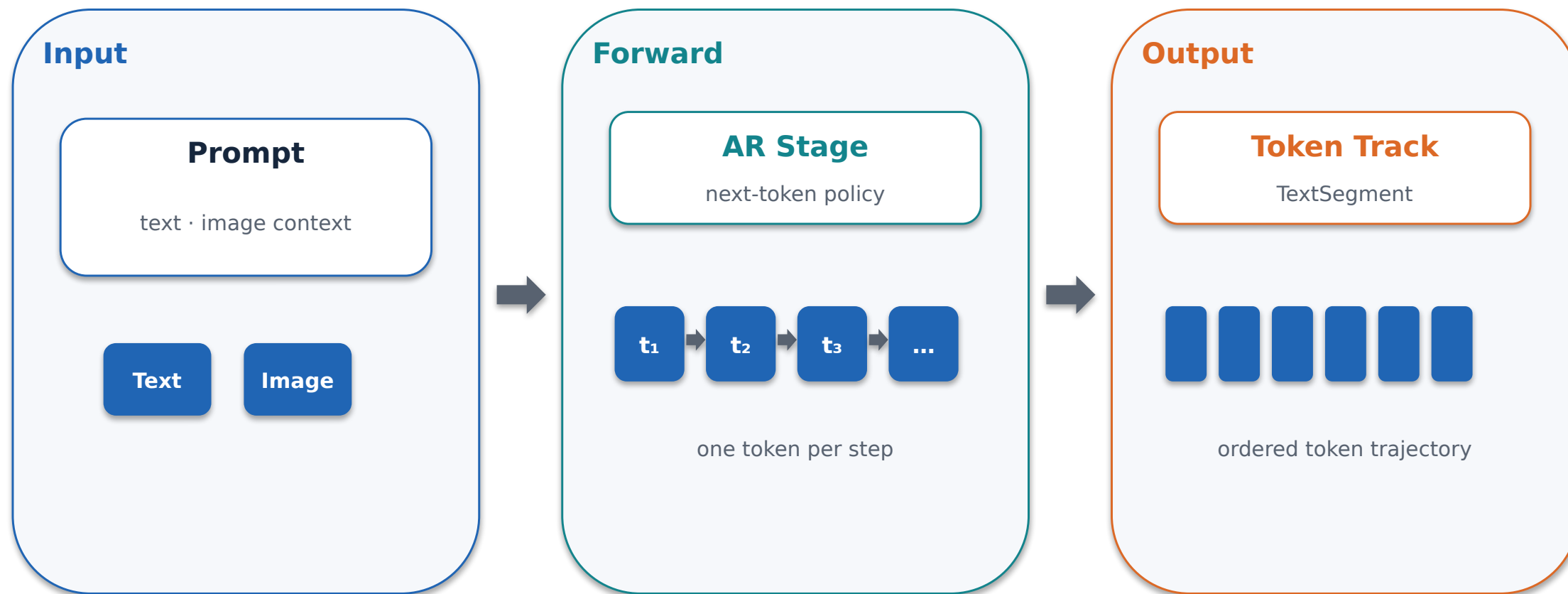
From One AR Path to Diverse Model Families

生成模型已从单一 AR token generation，扩展到 AR、Diffusion、Flow、共享 Unified Model 与多模型 Pipeline。模型族不同，forward、output 与训练路径也随之变化。



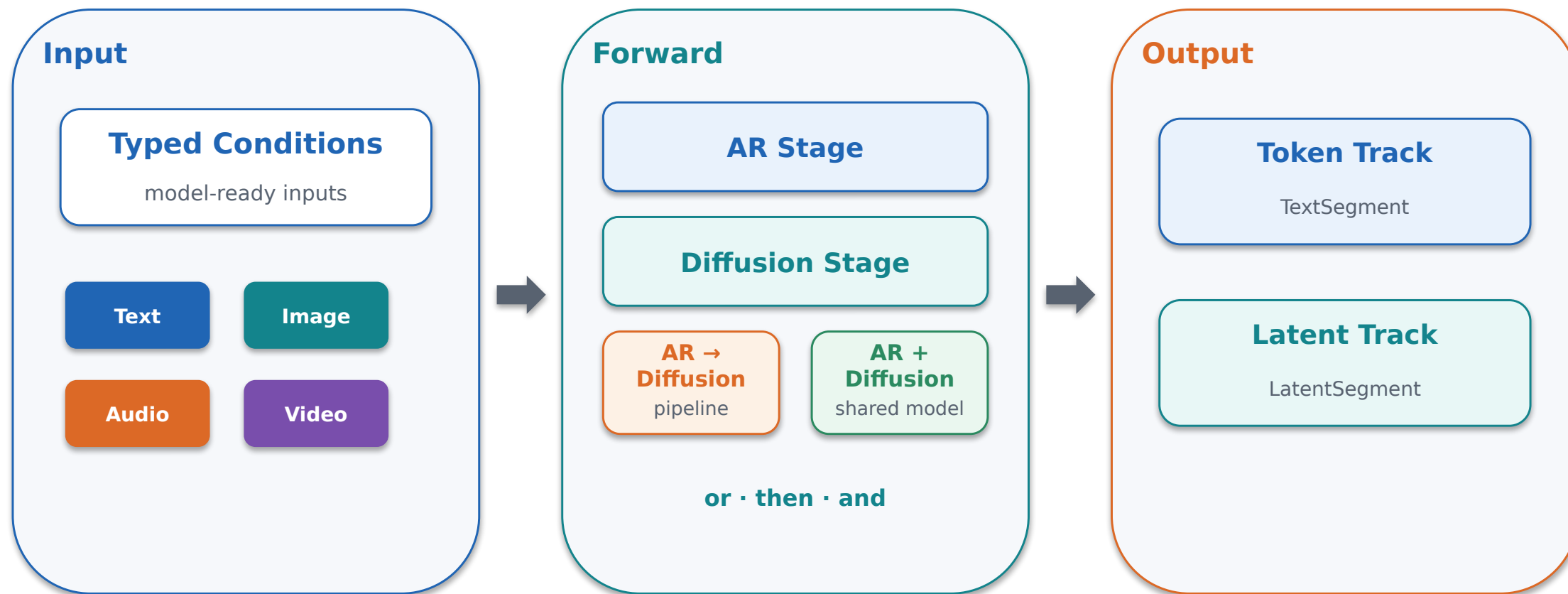
AR Framework: AR Stage Only

传统 AR 框架默认一条 AR Stage：Input、Forward 与 Output 都围绕 token generation 组织，最终形成有序 Token Track。



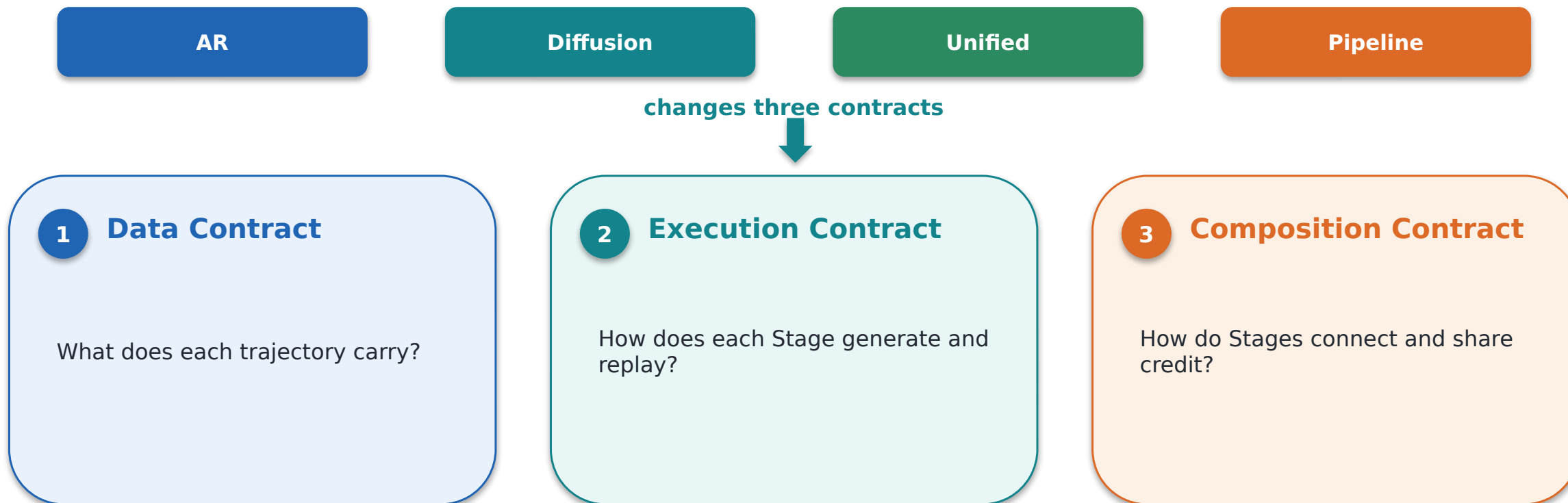
Unified Framework: Compose AR and Diffusion

统一框架必须支持 AR 与 Diffusion 的单独、串联和共享组合，并让 Token Track 与 Latent Track 使用同一训练抽象。



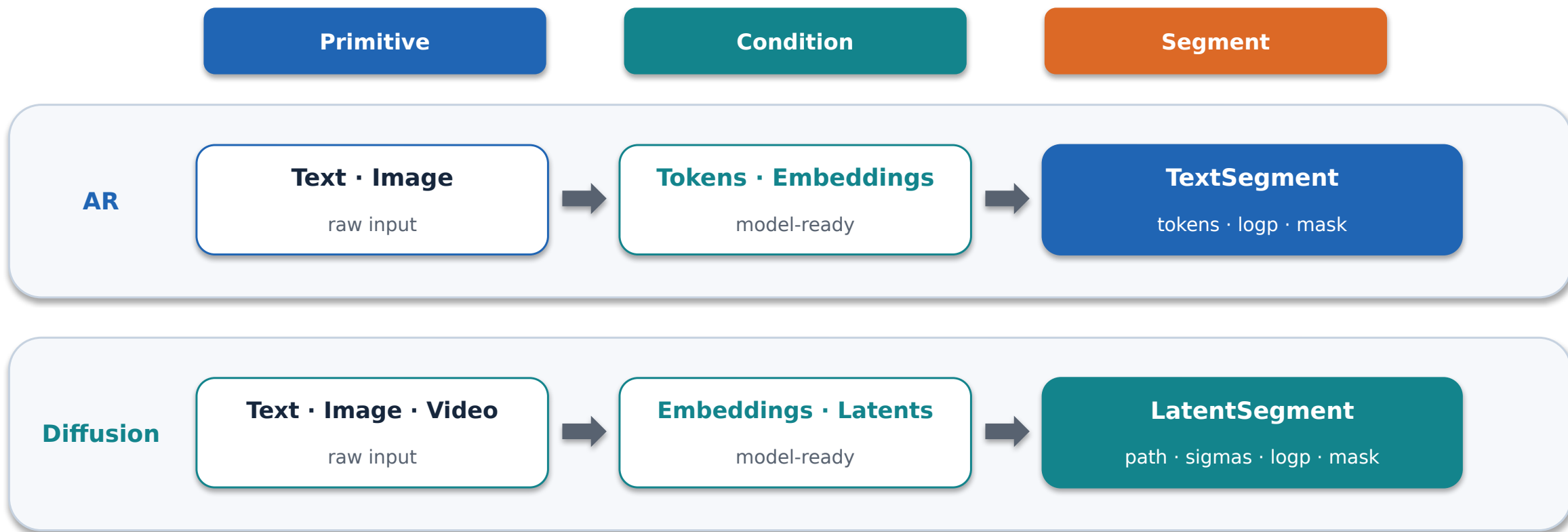
Model Diversity Changes Three RL Contracts

模型族变化不仅改变 forward，也改变三类 RL 契约：trajectory 承载什么、policy 如何 generate 与 replay，以及多个 Stage 如何连接并分配 credit。



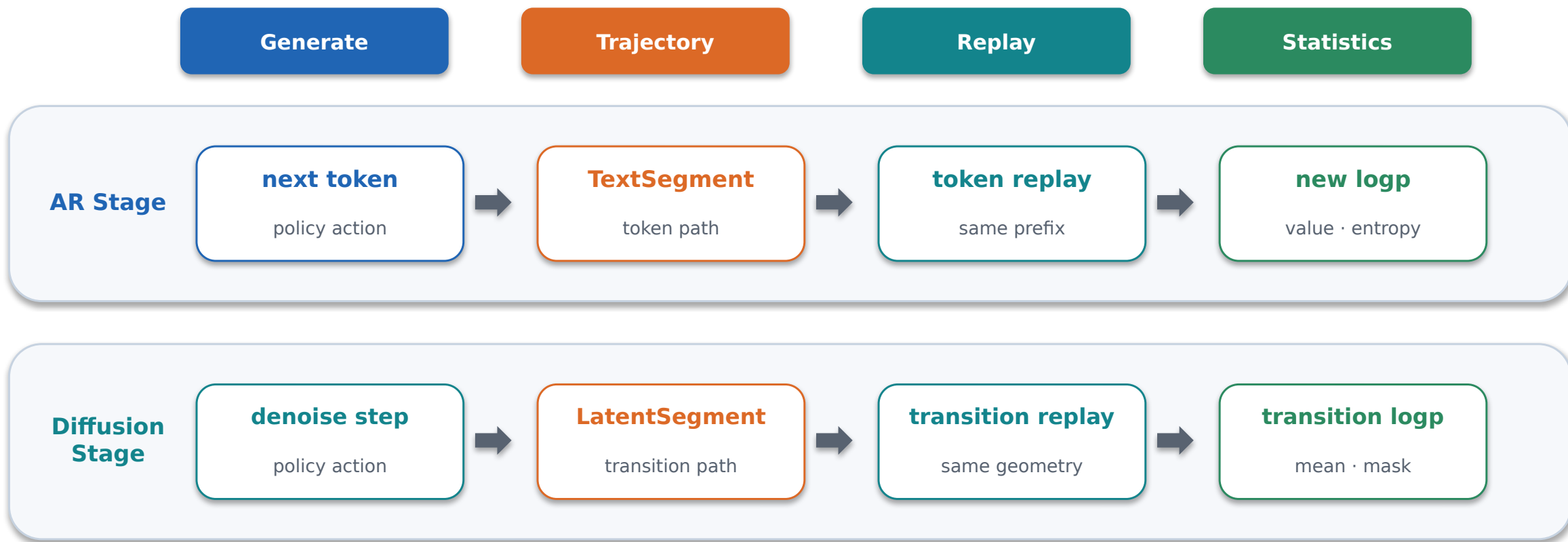
Data Contract: Primitive → Condition → Segment

Data Contract 定义每个 Stage 使用的数据类型。UniRL 用 Primitive、Condition 与 Segment 表达原始输入、模型条件和可训练轨迹，同时覆盖 AR 与 Diffusion。



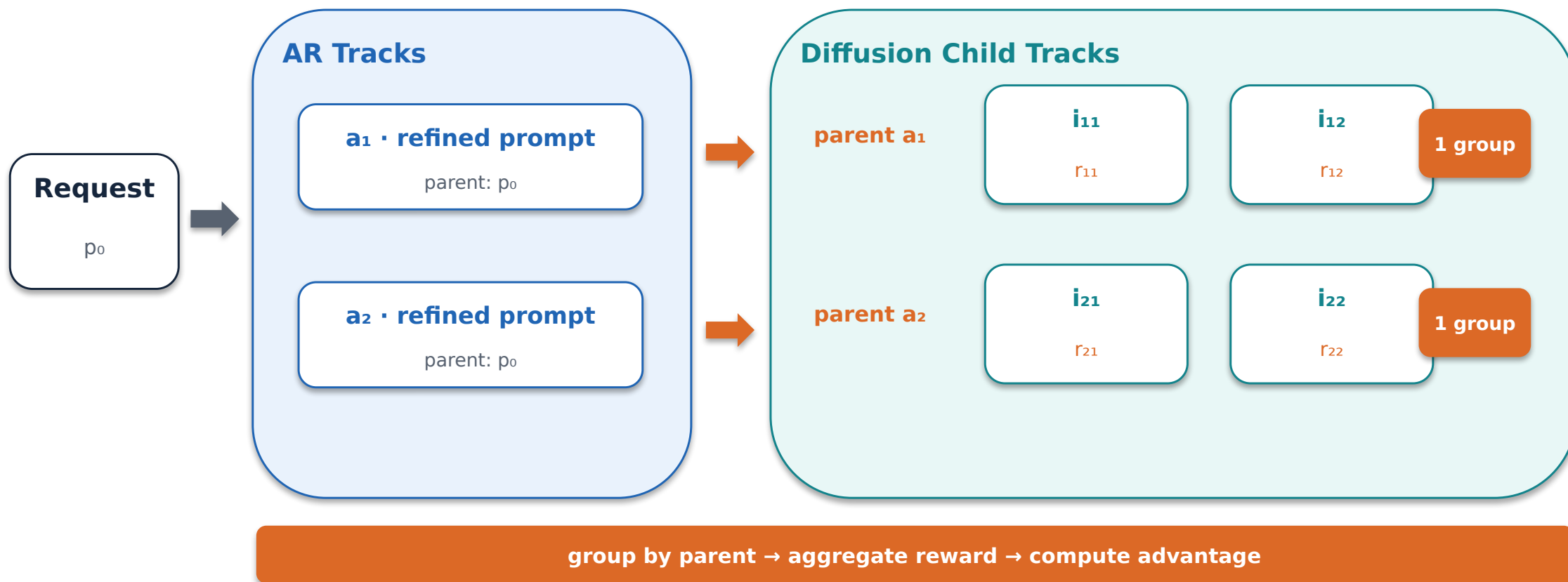
Execution Contract: Stage Owns Generate and Replay

Execution Contract 定义 Stage 如何 generate 与 replay。框架管理统一 lifecycle；ARStage 和 DiffusionStage 仅实现各自的 action loop、replay 与 policy statistics。



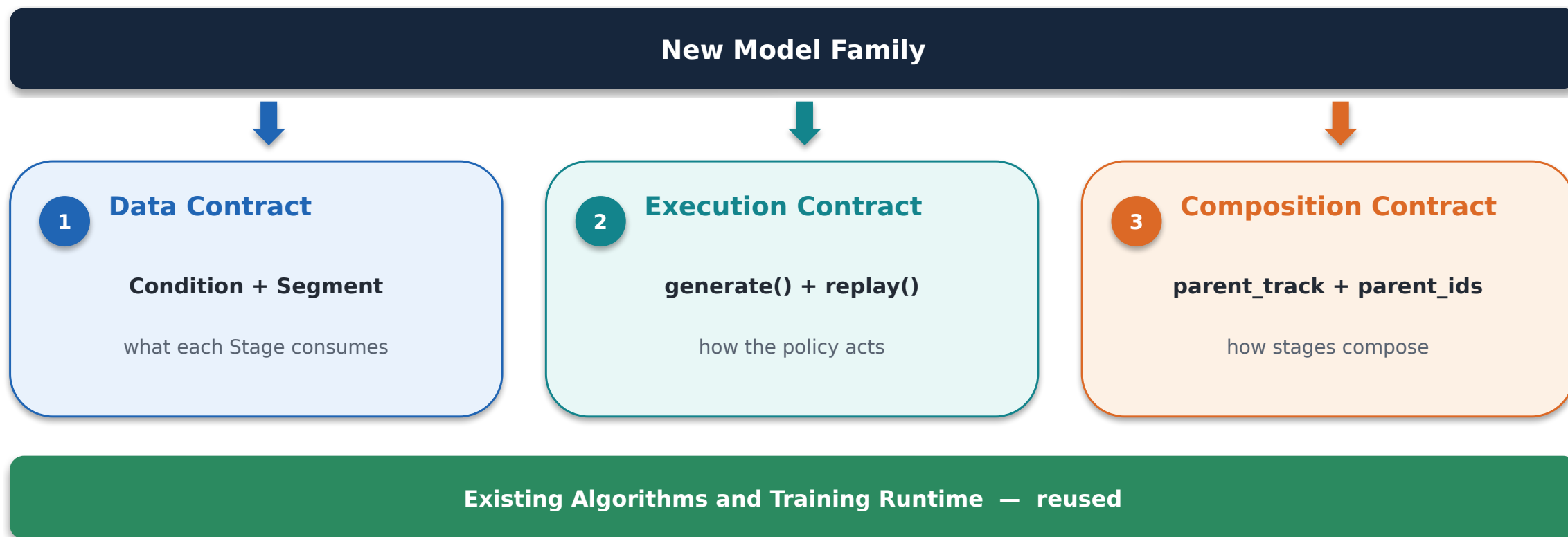
Composition Contract: Connect Trajectories and Credit

Composition Contract 定义多个 Stage 的 trajectory 与 credit 如何连接。RolloutTrack 保存局部轨迹，parent_track 和 parent_ids 将它们组成可分支、可聚合 reward 的轨迹树。



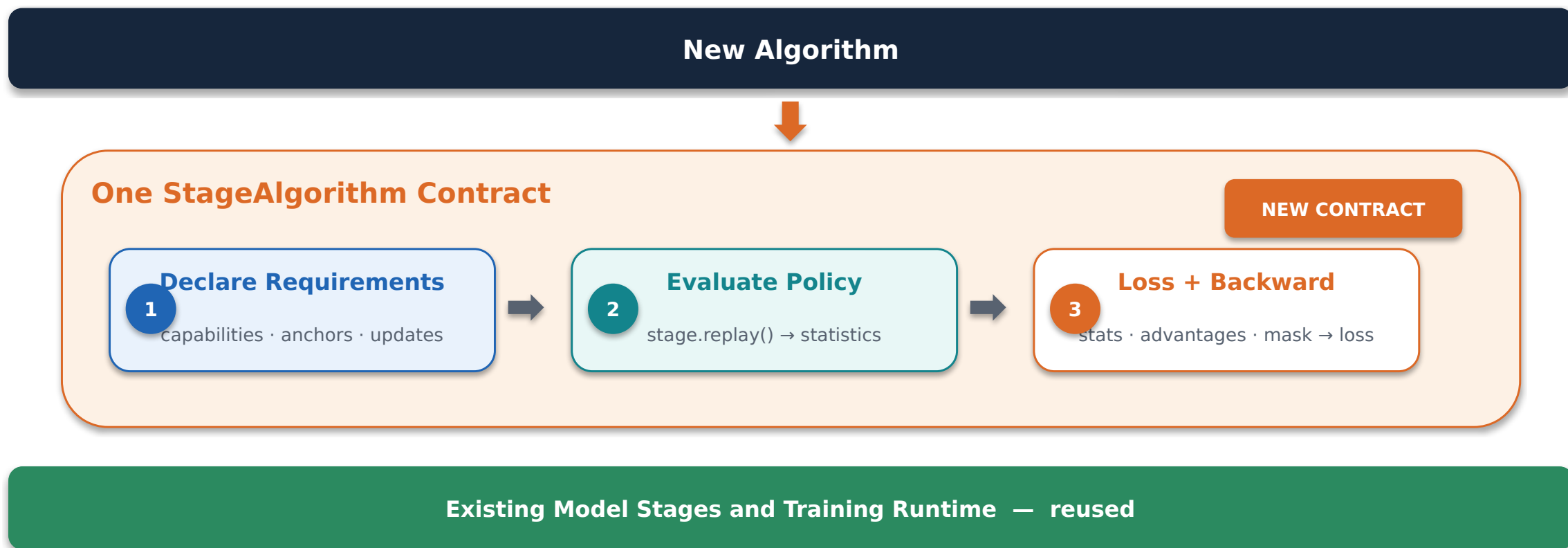
Adding a Model Family: Define Three RL Contracts

新模型族只需向 RL 框架说明三件事：训练数据如何表示、策略如何生成与重放、多个 Stage 如何连接 trajectory 与 credit。其余算法与训练流程保持复用。



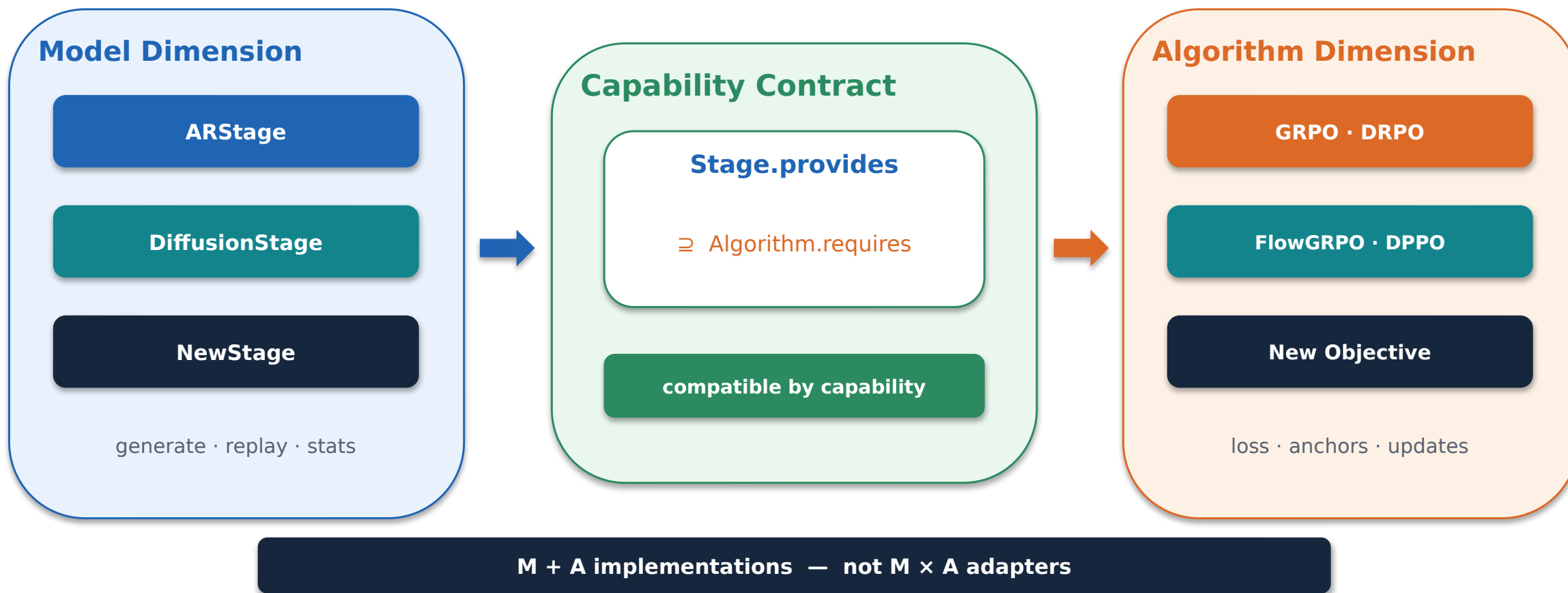
Adding an Algorithm: Define One StageAlgorithm Contract

新增算法只需定义一个 StageAlgorithm contract：声明所需能力与训练约束，复用兼容 Stage 的 replay，并在同一 autograd 域内完成 objective 与 backward。模型 Stage、TrainStack 与 runtime 保持复用。



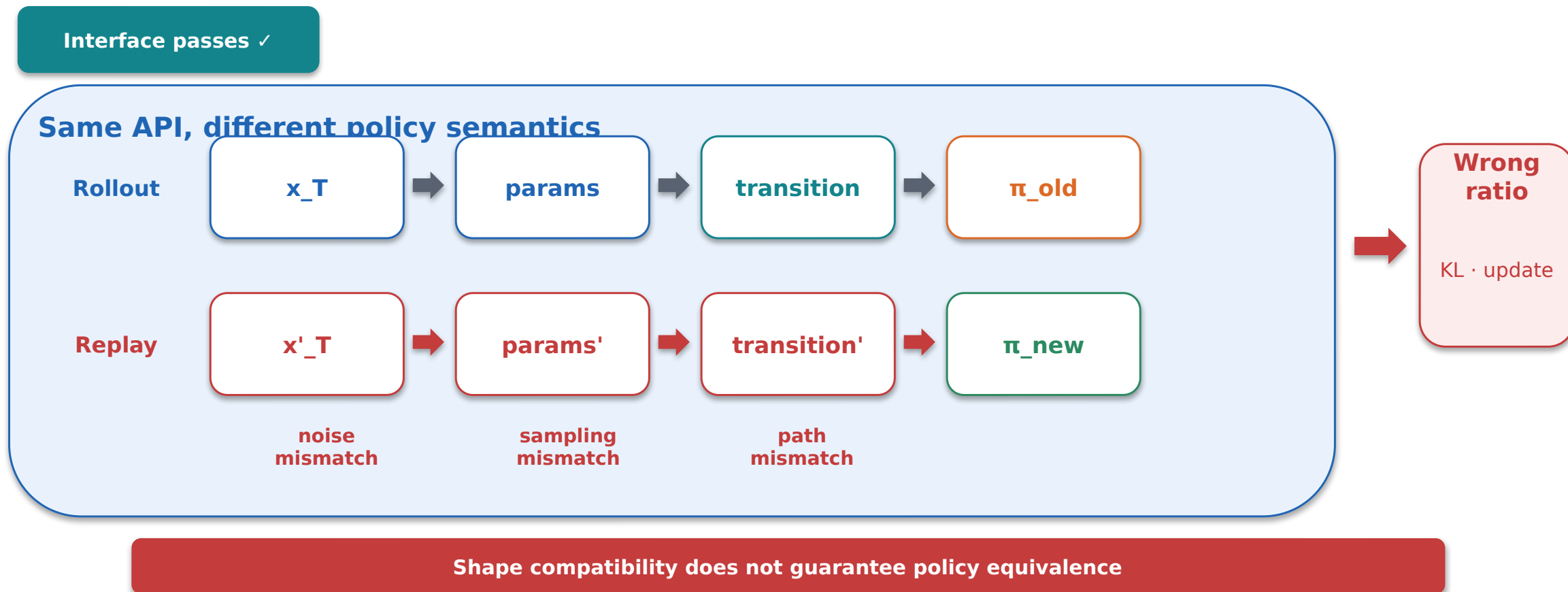
Core Advantage 1 — Model × Algorithm Independence

UniRL 将模型侧 generate 和 replay 封装在 Stage，将优化目标和训练要求封装在 StageAlgorithm。两者通过 capability contract 独立扩展，避免逐个适配 Model × Algorithm 组合。



Compatible Interface, Wrong Policy

AR replay 通常只需在相同 token prefix 上重算 log-prob；Diffusion 还必须复现 initial noise、sampling parameters、transition path 与 policy reference。任一语义漂移，ratio 与 KL 都会失真。



Core Advantage 2 — Correctness by Contract

UniRL 将 rollout-replay consistency 固化为四个可测试不变量：初始噪声可重建、transition 定义唯一、 π_{old} 更新期间固定、old/new 使用相同 replay geometry。

ROLLOUT SEMANTICS = REPLAY SEMANTICS

1

Same Initial Noise

NoiseRecipe $\rightarrow$ same x_T

2

One Transition

generate and replay share math

3

Frozen π_{old}

one anchor across updates

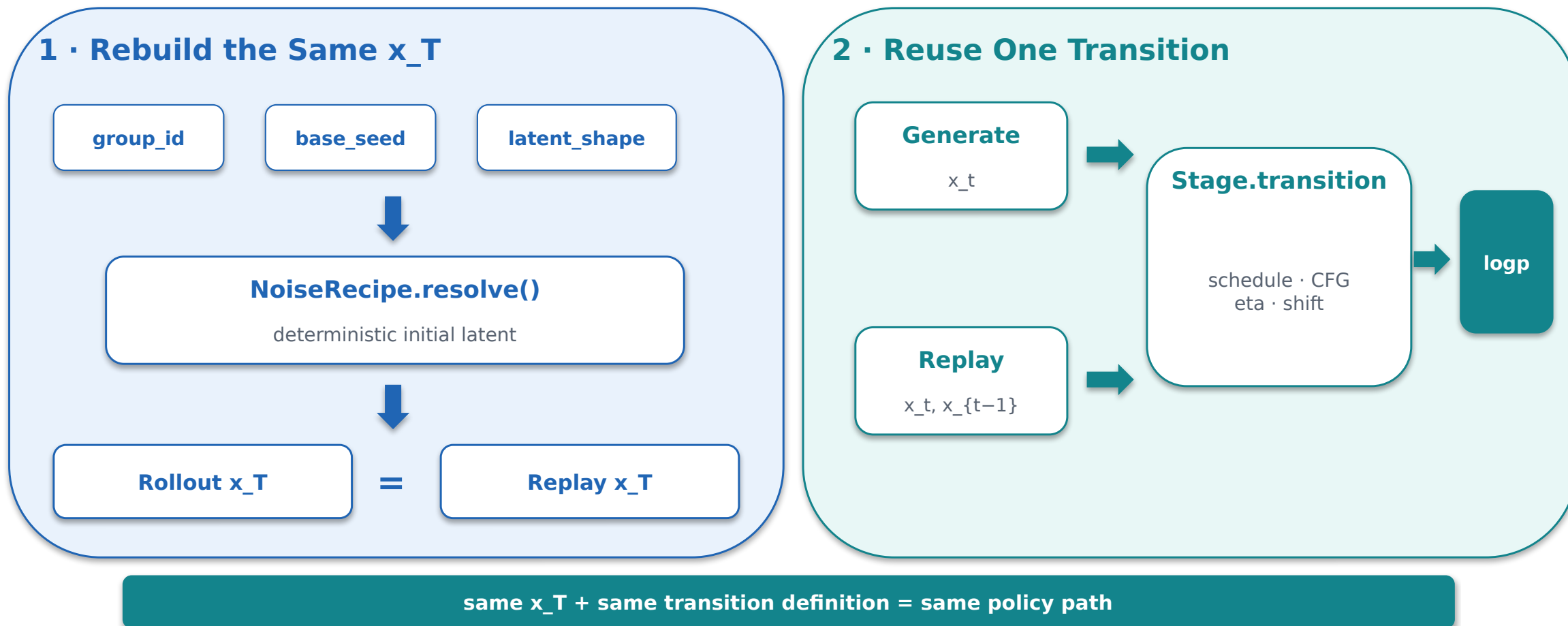
4

Same Geometry

same slices and precision

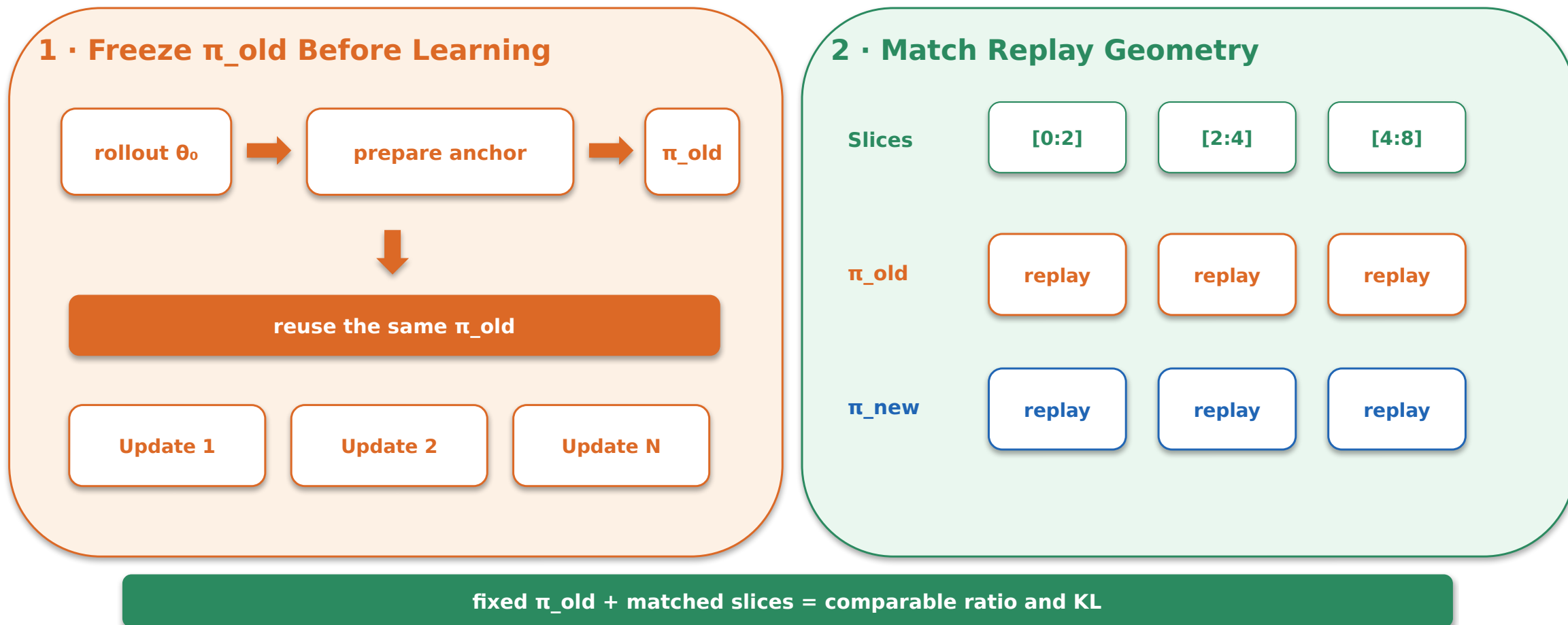
Correctness: Reconstruct the Same Policy Path

Diffusion policy 由完整生成路径定义，而不只是最终输出。UniRL 用 NoiseRecipe 重建同一 x_T ，并让 generate 和 replay 共用 Stage 中唯一的 transition 实现。



Correctness: Freeze a Comparable Reference

PPO-style objective 需要稳定的 π_{old} 和可比较的 old/new statistics。UniRL 在学习前冻结 anchor，并让 old/new 始终使用相同 mini/micro slices。



One New Modality, Many New Dependencies

LTX 从纯视频扩展到视频+音频。以 `decoded_audio`、`audio_sample_rate` 与 `aux_latents` 为例，少量新字段会同时引入 Audio Reward、AV Sync Reward 和 audio-aware replay 等新依赖。

Before · Video

```
@dataclass
class VideoTrack:
    decoded_video: Tensor
    video_latents: Tensor

    video_reward <- decoded_video
    train        <- video_latents
```



After · Video + Audio

```
@dataclass
- class VideoTrack:
+ class AVTrack:
    decoded_video: Tensor
+   decoded_audio: Tensor
+   audio_sample_rate: int
    video_latents: Tensor
+   aux_latents: Tensor

+   audio_reward <- decoded_audio
+   av_sync      <- decoded_video, decoded_audio
    train        <- video_latents
+   train.inputs += aux_latents
```


Explicit Task-Column Wiring Spreads Changes

以下以概念伪代码表示：在显式 Task-Column 架构中，除生产者与消费者外，通常还需同步维护 Queue schema 与 dependency table。

CHANGE 1 · Producer Output

```
+ queue.put('decoded_audio', audio)
+ queue.put('audio_sample_rate', rate)
+ queue.put('aux_latents', audio_path)
```

CHANGE 2 · Queue Columns

```
+ columns += ['decoded_audio',
              'audio_sample_rate',
              'aux_latents']
```

CHANGE 3 · Task Dependencies

```
+ audio_reward <- [
+   decoded_audio, audio_sample_rate]
+ av_sync <- [
+   decoded_video, decoded_audio]
+ train <- [video_latents, aux_latents]
```

CHANGE 4 · Consumer Reads

```
+ audio = queue.get(
+   'decoded_audio', 'audio_sample_rate')
+ sync = queue.get(
+   'decoded_video', 'decoded_audio')
+ train = queue.get(
+   'video_latents', 'aux_latents')
```

Core Advantage 3 — A Declarative Multimodal Data Plane

Typed trajectory 是数据面的语义 schema：生产者定义输出，消费者声明所需视图；UniRL 运行时递归生成 tensor-leaf projection 与 TensorRef transport plan。

User Code · Declare WHAT

```
def generate(...) -> RolloutTrack:
    return track

@consumes(
    'decoded_audio',
    'audio_sample_rate',
)
def audio_reward(track): ...

@consumes(
    'segment.latents',
    'segment.aux_latents',
    'conditions',
)
def train(track): ...
```



Runtime · Derive HOW

```
leaves = discover_tensor_leaves(track)

fields = consumer.required_fields
projected = project(track, fields)

refs = dehydrate(projected)
route(refs, destination=consumer)

# no hand-written transport schema
```

The Same Upgrade Becomes a Local Extension

在 LTX 视频+音频场景中，生产者扩展 typed output ，实际消费者声明所需视图；运行时生成投影，无需另行维护传输层 schema 与全局 dependency table 。

Producer · Add Fields Locally

```
return RolloutTrack(  
    decoded=video,  
    + decoded_audio=audio,  
    + audio_sample_rate=rate,  
    segment=LatentSegment(  
        latents=video_path,  
        + aux_latents=audio_path,  
    ),  
)
```

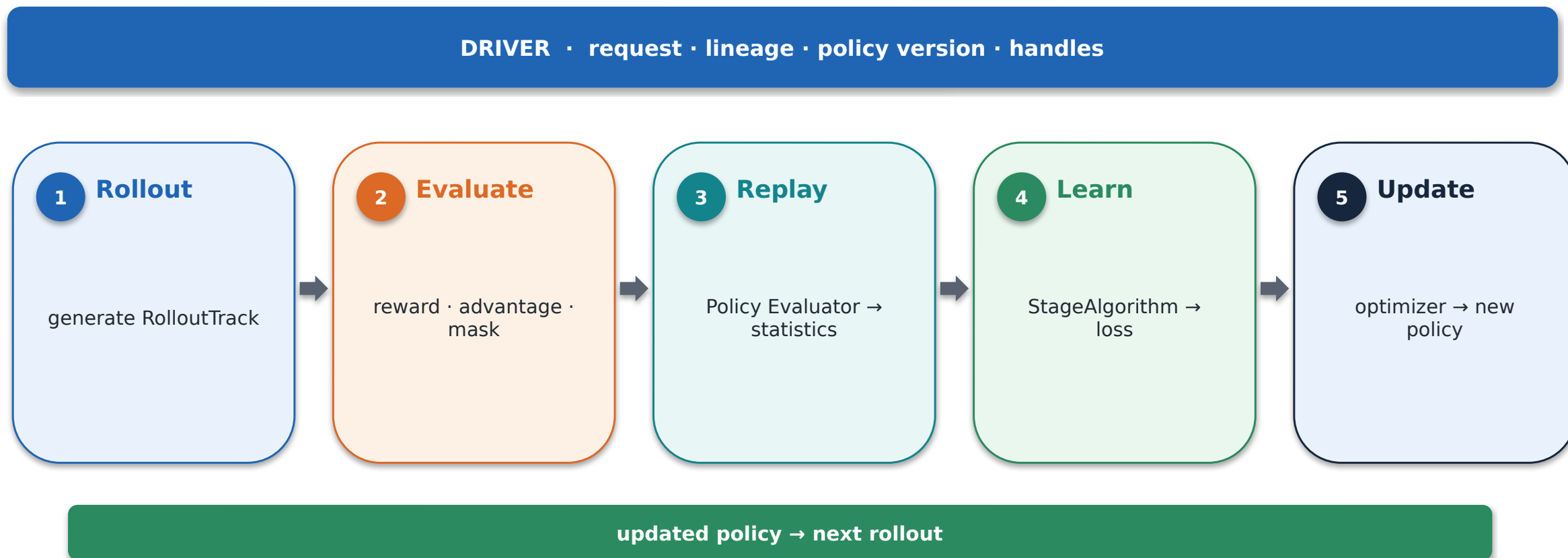


Consumers · Declare Their Views

```
+ @consumes(  
+     'decoded_audio', 'audio_sample_rate')  
+ def audio_reward(track): ...  
  
+ @consumes('decoded', 'decoded_audio')  
+ def av_sync_reward(track): ...  
  
@consumes('segment.latents',  
+         'segment.aux_latents',  
+         'conditions')  
def train(track): ...  
  
# no hand-written transport schema  
# no global dependency-table update
```

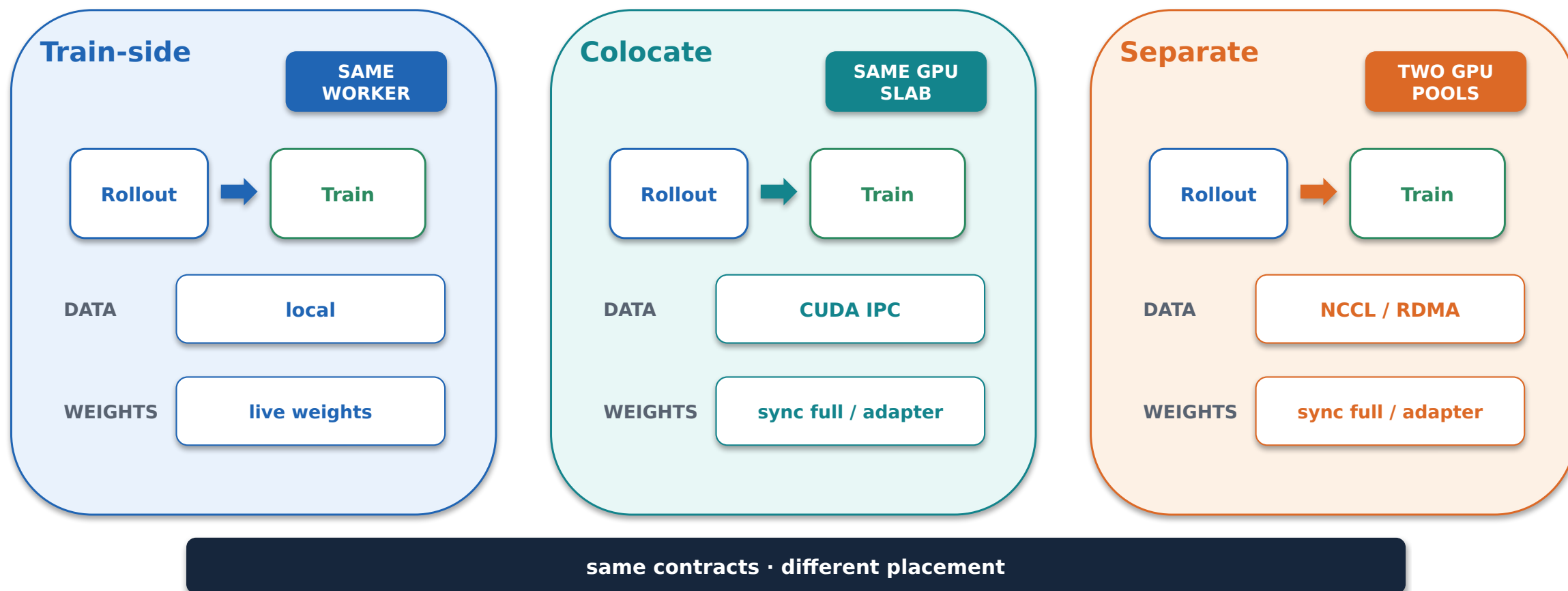
Runtime: From Rollout to Update

Driver 编排请求、lineage 与 policy version ; Rollout Worker 生成 RolloutTrack , Reward 路径计算 reward、advantage 与 mask , Train 路径 replay policy、计算 loss 并发布新权重。



One Runtime, Three GPU Topologies

Rollout、Reward 与 Train 的逻辑调用保持不变，Remote 仅根据 placement 选择 Train-side、Colocate 或 Separate。变化的是组件位置、数据路径和权重同步方式。



Shared Runtime Infrastructure

除三项核心优势外，UniRL 提供共享运行时能力，使 typed trajectory 在不同 batch、Worker 和部署拓扑下保持一致的执行语义。

Typed Batch Semantics



sample and stage boundaries preserved

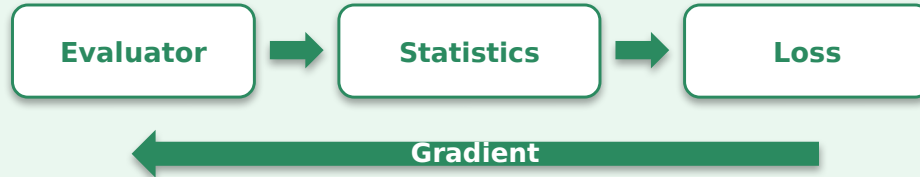
Flexible Placement



Topology-Aware TensorRef

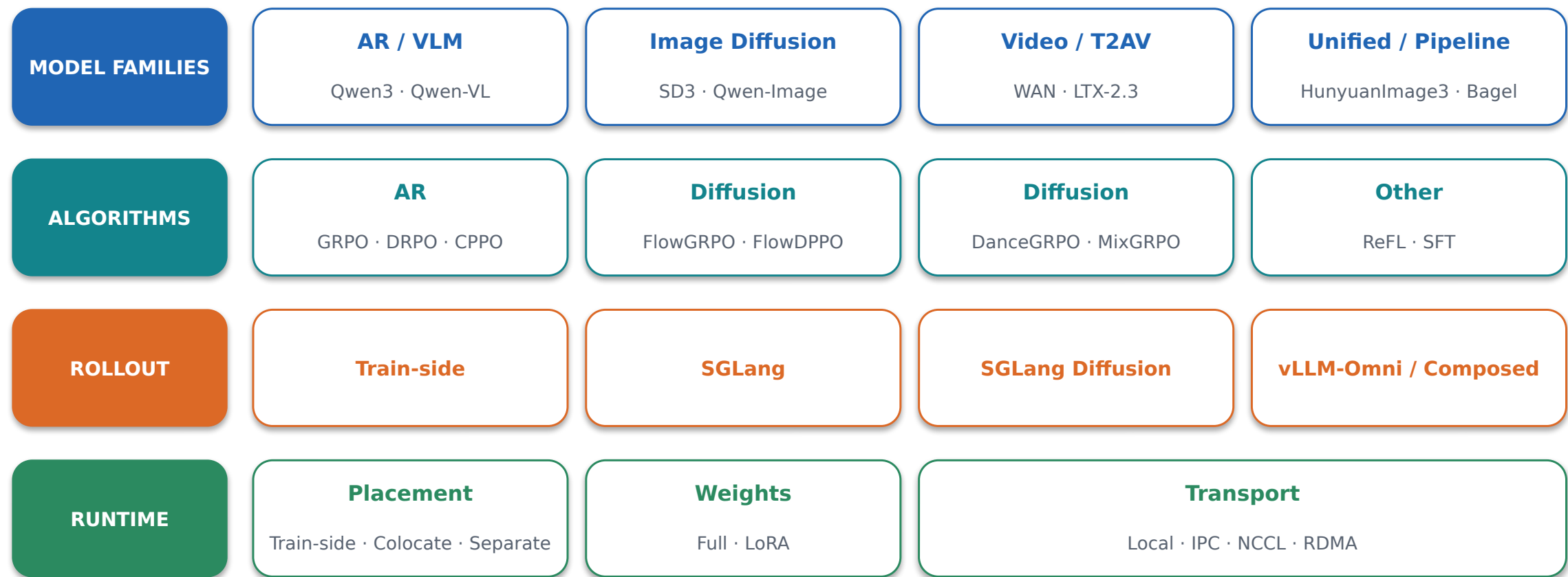


Distributed Autograd



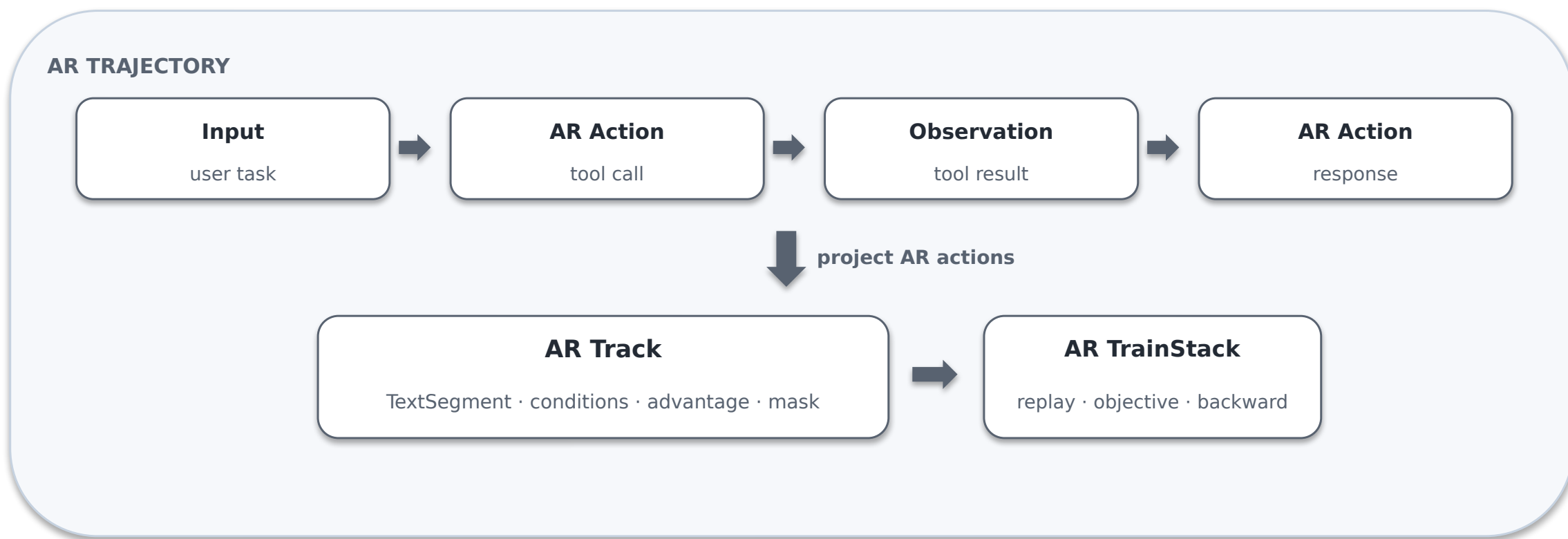
Current Coverage

UniRL 已覆盖 AR、图像与视频 Diffusion、音视频联合生成、共享 Unified Model 与多模型 Pipeline，并提供对应算法、rollout engine 和分布式模式。下图为代表性覆盖，不表示所有组合均已端到端验证。



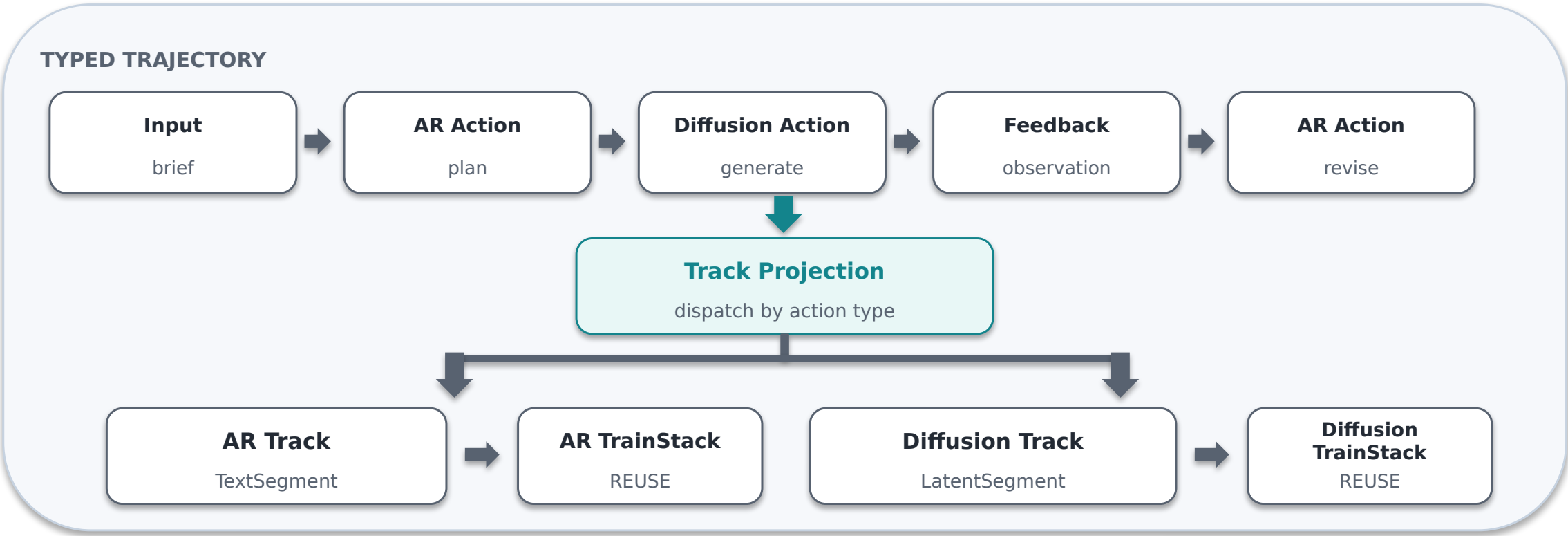
UniRL for Standard Agentic RL: One AR Track

传统 Agentic RL 只有 AR actions 可训练，因此一条 trajectory 对应一个 AR Track。



UniRL for AIGC Agentic RL: AR Track + Diffusion Track

同一条 typed trajectory 中，AR 与 Diffusion actions 按类型进入各自 Track，并复用已有训练栈。



UniRL for World Models: Real Track + Imagined Track

同一 typed episode 同时保留 real 与 imagined transitions ; Policy/Value 复用现有 Track , 只新增 provenance 与 World Model Learner 。

